

Guía Práctica: Sistema RAG y Base de Conocimiento

(Retrieval-Augmented Generation) con ChromaDB

Curso de Seguridad Informática

16 de mayo de 2026

1. Introducción

En nuestra Suite de Auditoría de Seguridad Informática, hemos implementado un sistema **RAG (Retrieval-Augmented Generation)**. Este actúa como el “segundo cerebro” o la memoria a largo plazo de nuestra herramienta.

En lugar de depender de búsquedas externas para resolver los mismos problemas repetidamente, nuestra base de datos (impulsada por ChromaDB) aprende de los errores específicos que cometemos durante el desarrollo y los guarda basándose en su **significado semántico**.

¿Qué significa esto?

A diferencia de una búsqueda tradicional que requiere coincidencias exactas de palabras (como hacer **Ctrl+F**), una base de datos vectorial entiende el contexto y la intención. Por ejemplo, si el RAG aprendió una solución para un error documentado como *“Fallo de conexión por puertos bloqueados”*, y tú le preguntas al sistema *“no me deja conectar al objetivo, creo que un firewall me detiene”*, el RAG sabrá que hablas del mismo problema y te entregará la respuesta correcta, ¡incluso si no usaste ni una sola palabra idéntica!

2. Arquitectura del Sistema

Nuestro flujo de conocimiento consta de los siguientes componentes:

- **ChromaDB:** La base de datos vectorial local (almacenada en `chroma_db/`).
- **Archivos Markdown (.md):** Archivos legibles ubicados en `docs/` que actúan como la “fuente de la verdad”.
- `knowledge_db.py`: Herramienta CLI para consultar e inyectar conocimientos.
- `exportar_errores_json.py`: Script que detecta fallos en el orquestador automáticamente.

3. ¿Cómo alimentar el RAG?

Existen dos formas de enseñar a nuestra base de datos para que el conocimiento del grupo crezca de forma colaborativa:

3.1. 1. Alimentación Manual (Conocimiento Teórico o de Entorno)

Cuando tu grupo resuelva un problema (ej. configuración de Docker, errores de Git o fallos de Python), documenten la solución:

1. Editen el archivo Markdown correspondiente (ej. docs/python.md).
2. Estructuren su nuevo aporte creando un título usando doble almohadilla (##).
3. Guarden el archivo y actualicen la memoria ejecutando en la terminal:

Listing 1: Actualizar colección de Python

```
1 python knowledge_db.py -c python -u
```

Nota: La bandera -c define la colección y -u ordena la actualización (update).

3.2. 2. Flujo Automático (Feedback Loop de la Suite)

Si un módulo de la suite (ej. osint.py) falla durante la ejecución de auditoria.py, el error se guarda en historial_auditoria.json.

1. Extrae los fallos recientes del orquestador ejecutando:

```
1 python exportar_errores_json.py
```

2. Esto actualizará automáticamente el archivo docs/auditoria.md, dejando un espacio que dice **La Solución (Pendiente)**.
3. Reúnete con tu equipo, redacten la solución técnica allí y compilen el aprendizaje en el RAG:

```
1 python knowledge_db.py -c auditoria -u
```

4. ¿Cómo consultar el RAG?

Para solucionar un problema, pregúntale a la base de datos usando lenguaje natural (sin importar si recuerdas las palabras exactas):

Listing 2: Consulta semántica en lenguaje natural

```
1 python knowledge_db.py -c auditoria -q "falla el modulo osint por  
un rate limit"
```